

CS2033: Data Communication and Networking

Transmission Errors 2

Sunimal Rathnayake

Department of Computer Science and Engineering

University of Moratuwa

sunimal [at] cse [.] mrt [.] ac [.] lk

Information Redundancy - Coding

- A data word with d bits is encoded into a codeword with c bits - $c > d$
- To extract original data - c bits must be decoded
- If the c bits do not constitute a valid codeword an error is detected
- For certain encoding schemes - some types of errors can also be corrected

Separability of a Code

- A code is separable if it has separate fields for the data and the code bits
- Decoding consists of disregarding the code bits
- The code bits can be processed separately to verify the correctness of the data
- A non-separable code has the data and code bits integrated together - extracting the data from the encoded word requires some processing

Parity Codes

- The simplest separable codes are the parity codes
- A parity-coded word includes d data bits and an extra bit which holds the parity
- In even parity code - the extra bit is set so that the total number of 1's in the $(d+1)$ -bit word (including the parity bit) is even
- In even odd parity code - the extra bit is set so that the total number of 1's in the $(d+1)$ -bit word (including the parity bit) is odd

Parity Code Example

- Even parity of 1s

Data							Parity
1	0	0	1	1	1	0	0
0	0	1	1	1	0	0	1
1	1	1	0	1	1	1	0

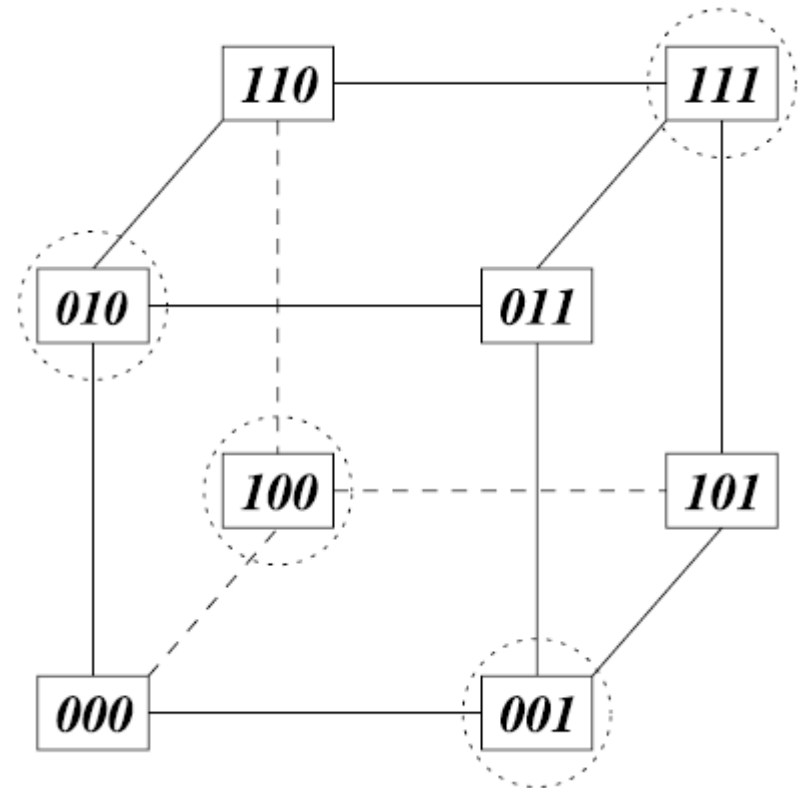
2-Dimensional Parity Example

- Even parity of 1s
- Longitudinal Redundancy Check(LRC)/Vertical Redundancy Check(VRC)
- Identifies a unique erroneous bit

Data							VRC
1	0	0	1	1	1	0	0
0	0	1	1	1	0	0	1
1	1	1	0	1	1	1	0
1	1	1	0	1	1	1	0
0	0	1	1	1	0	0	1
0	0	1	0	1	0	1	1
0	0	1	1	1	0	0	1
1	0	0	0	1	1	1	0
LRC							

Hamming Distance

- The Hamming distance between two codewords - the number of bit positions in which the two words differ
- Two words in this figure are connected by an edge if their Hamming distance is 1



Hamming Code

- Developed based on hamming distance concept
- Assigns multiple parity bits to cover each bit of data
- Many different hamming code schemes exist
- For simplicity we will only look at (7,4) Single Error Correcting (SEC) Hamming Code
 - 7-bit code (4 data bits + 3 parity bits)

(7,4) SEC Hamming Code

- Can correct 1-bit errors
- How to calculate number of parity bits?
 - $2^r \geq m + r + 1$
where r = number of parity bits
 m = message bits (data)
- If $m = 4$; according to above inequality $r = 3$

Parity bits placement

- In the resulting codeword, parity bits can be placed at different places. This placement affects how easy/difficult to detect and correct errors
- One way is to place parity bits at places corresponding to powers of two

$m_7m_6m_5p_4m_3p_2p_1$ ← Bit position

- E.g. say $m = 1001$
 $100p_41p_2p_1$

Parity bit coverage

7 6 5 4 3 2 1
1 0 0 P_4 1 P_2 P_1

Sender

	P_4	P_2	P_1
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

$P_1 \rightarrow 1, 3, 5, 7 \rightarrow P_1 101$

$P_2 \rightarrow 2, 3, 6, 7 \rightarrow P_2 101$

$P_4 \rightarrow 4, 5, 6, 7 \rightarrow P_4 001$

Assuming Even Parity;

$P_1 = 0$ $P_2 = 0$ $P_4 = 1$

Detecting Error

Receiving

1 0 0 1 1 0 0 ← original

Suppose received. 1 1 0 1 1 0 0

Compute parity;

$$P_1 \rightarrow 1, 3, 5, 7 \rightarrow 0 \oplus 0 \oplus 1 = 1$$

$$P_2 \rightarrow 2, 3, 6, 7 \rightarrow 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

$$P_4 \rightarrow 4, 5, 6, 7 \rightarrow 1 \oplus 0 \oplus 1 \oplus 0 = 0$$

	P_4	P_2	P_1
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Correcting Error

Received

1 0 0 1 1 0 0 $\leftarrow$ original

Suppose
received.

1 1 0 1 1 0 0

Compute parity;

$$P_1 \rightarrow 1, 3, 5, 7 \rightarrow 0101 = 0$$

$$P_2 \rightarrow 2, 3, 6, 7 \rightarrow 0111 = 1$$

$$P_4 \rightarrow 4, 5, 6, 7 \rightarrow 1011 = 1$$

$$P_4 P_2 P_1 = 110$$

decimal

6

* 6th bit is

the wrong bit.

Cyclic Redundancy Checksum (CRC)

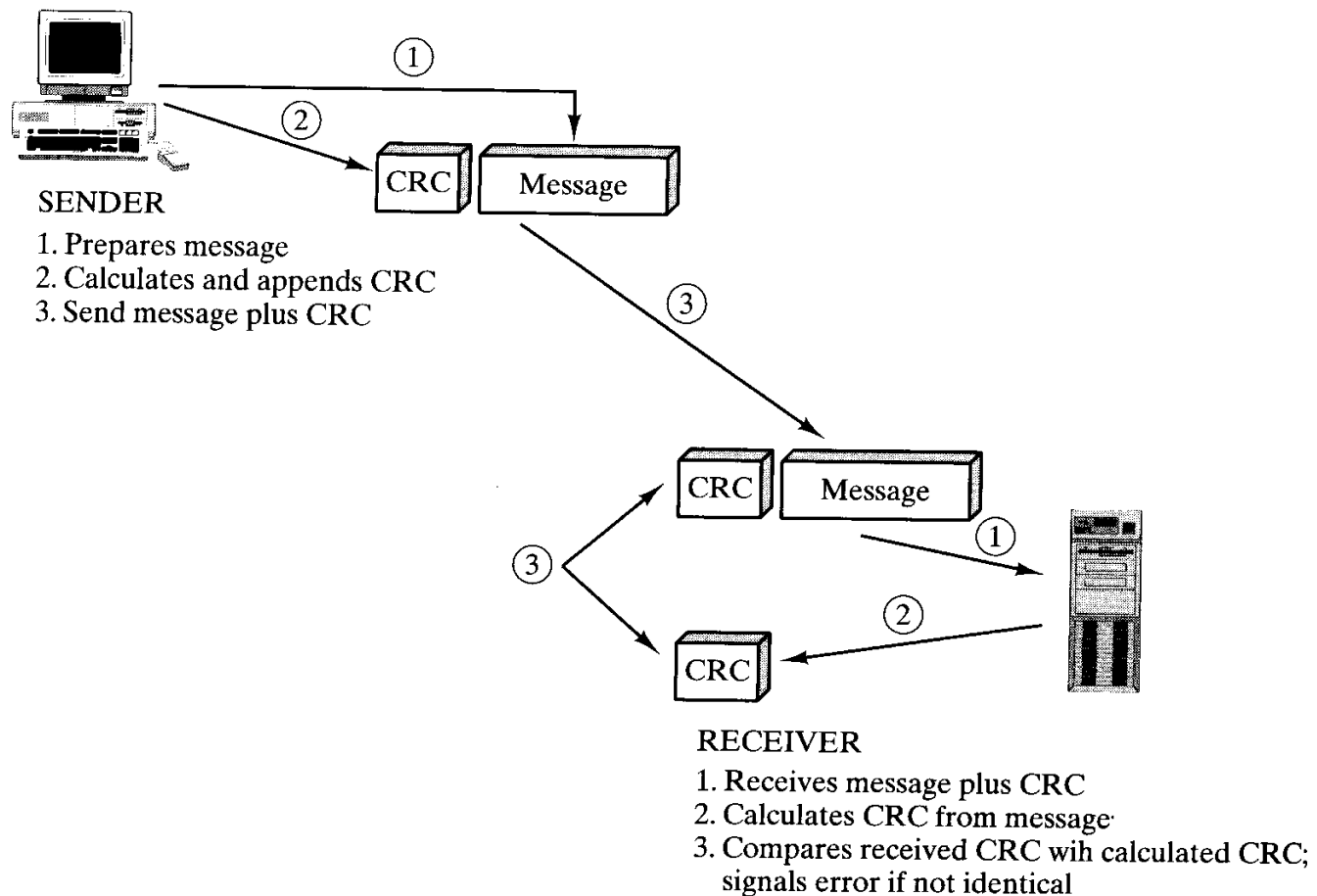
- A non-separable code
- The CRC error detection method treats the packet of data to be transmitted as a large polynomial.
- The transmitter takes the message polynomial and using polynomial arithmetic, divides it by a given generating polynomial.
- The quotient is discarded but the remainder is “attached” to the end of the message (remainder (mod) arithmetic)

Cyclic Redundancy Checksum (CRC)

- The message (with the remainder) is transmitted to the receiver.
- The receiver divides the message and remainder by the same generating polynomial.
- If a remainder not equal to zero results, there was an error during transmission.
- If a remainder of zero results, there was no error during transmission.

Cyclic Redundancy Checksum (CRC)

FIGURE 5.14 Error Detection Using CRC.



CRC Example

Message = 101101

CRC Polynomial = $x^3 + x^2 + 1$

CRC Polynomial bits = 1101

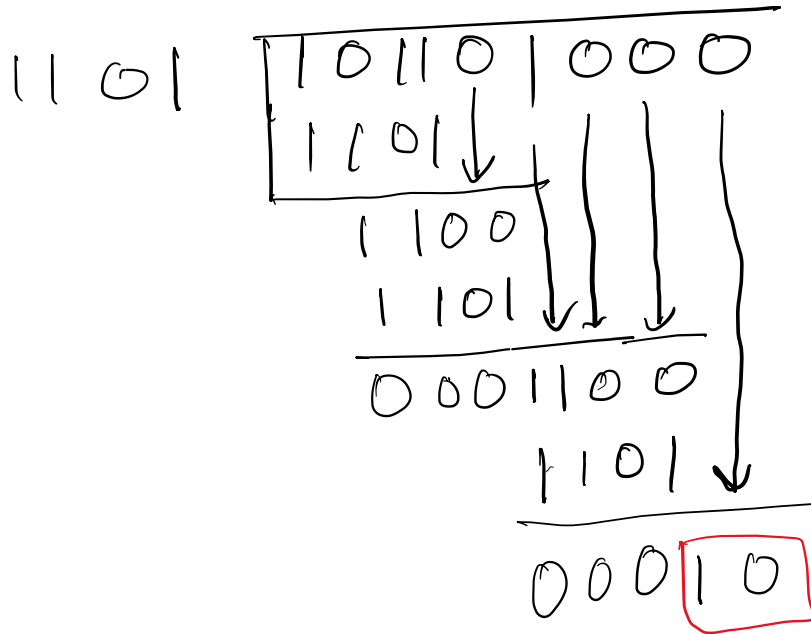
of CRC bits = 3 (1 bit less than polynomial bits)

How to calculate?

Modulo-2 long division (XOR)

Remainder is the CRC

Cyclic Redundancy Check



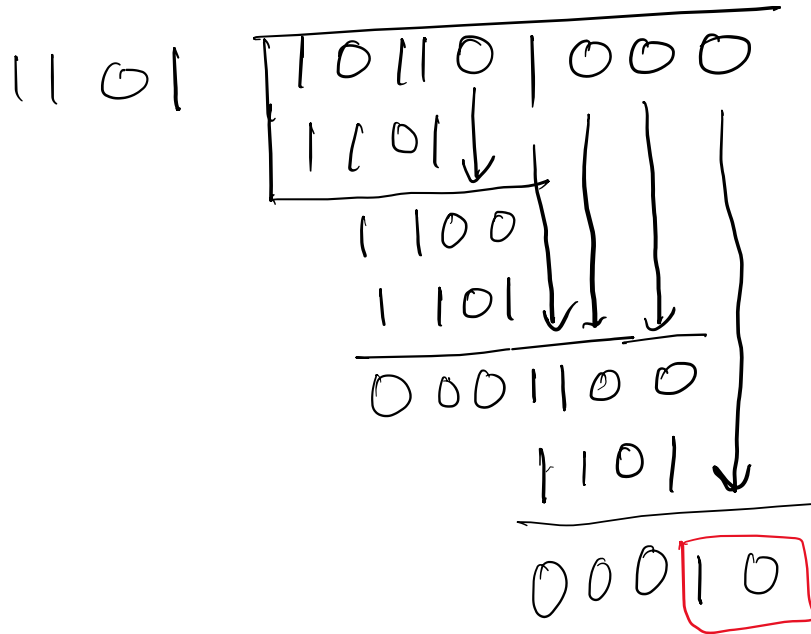
Remainder = 10

But we want 3 bits.

Thus $\Rightarrow$ 010 = CRC bits

Message + CRC is sent to other side.

Cyclic Redundancy Check



* At recipient,
divide Msg + CRC with
the same polynomial.
If remainder $\neq 0$,
ERROR

Remainder = 10

But we want 3 bits.

Thus $\Rightarrow$ 010 = CRC bits

Message + CRC is sent to other side.



Summary

- Information Coding
- Parity codes
- Hamming codes
- CRC